CAREERCOMPASS AI: PLACEMENT INTELLIGENCE AND PROFILE GAP ANALYSIS SYSTEM

A PROJECT BASED LEARNING (PBL) REPORT

Submitted by

R SABARISH KUMAR

N DINESH SAI KARTHIK

Submitted in partial fulfilment of the requirements

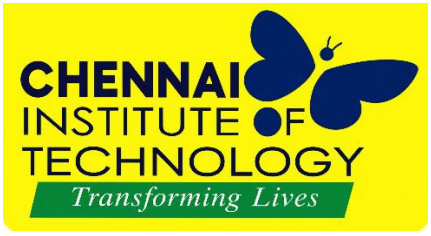
for the

Project-Based Learning component of Machine Learning

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI



Affiliated to Anna University, Chennai

(Autonomous)

OCTOBER 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

IM1. To creates next generation leaders by effective teaching learning methodologies and in still Scientifics Park in them to meet the global challenges.

IM2. To transform lives through deployment of emerging technology, novelty and sustainability.

IM3. To inculcate human values and ethical principles to cater the societal needs.

IM4. To contributes towards the research ecosystem by providing a suitable

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

SIRAGU

Mission of the Department:

DM1. To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.

DM2. To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.

DM3. To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

BONAFIDE CERTIFICATE

This is to certify that the Project–Based Learning report titled "CAREERCOMPASS AI: PLACEMENT INTELLIGENCE AND PROFILE GAP ANALYSIS " is a Bonafide record of work carried out by [Student Name(s), Register No(s).] of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor–guided Project-Based Learning (PBL) component of the Machine Learning course during the academic year [2026–2027] under my supervision.

SIRAGU

SIGNATURE

Dr. S. Pavithra, M.E., Ph.D.

Professor and Head,

Dept. of Computer Science and Engineering

Chennai Institute of Technology,

Chennai – 69.

SIGNATURE

<<Name>>

MENTOR

<<Designation>>

Dept. of Computer Science and Engineering

Chennai Institute of Technology,

Chennai – 69.

Submitted for the final review held on ……

Internal Examiner

DECLARATION

I/We jointly declare that the PBL report on **"CareerCompass AI: Placement Intelligence and Profile Gap Analysis System"** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **"ANNA UNIVERSITY, CHENNAI"** for the requirement of Degree of **BACHELOR OF ENGINEERING.** This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of **COMPUTER SCIENCE AND ENGINEERING.**

SIRAGU

Signature

R SABARISH KUMAR

N DINESH SAI KARTHIK

Place: Chennai

Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honorable Chairman **SHRI. P. SRIRAM** for providing immense facilities at our institution.

We are very proudly rendering our thanks to our Principal **Dr.A.RAMESH M.E**, **Ph.D.,** for the facilities and the encouragement given by him to the progress and completion of our project.

We would like to express special thanks of gratitude to our Dean **Dr. V. SRINIVASA RAO, M.E., Ph.D.,** who has been the key spring of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr. S. PAVITHRA M.E, Ph.D.,** for her effective leadership, encouragement and guidance in the project.

We would like to extend our thanks to the Project Co-ordinator **K HARI .,** Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from the class advisors **NISHANTHINI** and **JIIVA BHARATHI .,** of the Department of Computer Science and Engineering and others for providing valuable suggestions and for the successful completion of the project.

R SABARISH KUMAR (2104251040734)

N DINESH SAI KATHIK (2104251040601)

ABSTRACT

CareerCompass AI is a placement-intelligence and profile-gap-analysis system that connects students, industry recruiters and academic administrators around a single, data-driven view of placement readiness. Final-year engineering students frequently cannot judge, in concrete terms, how their academic and extra-curricular profile compares against what recruiters actually screen for, which leads to mis-targeted applications and avoidable rejections. This project builds the machine-learning core of that system: a supervised classifier trained on a college placement dataset (9,939 records, 10 columns, features including CGPA, IQ, communication skills, projects completed and internship experience) to predict placement outcome and to rank the features driving that prediction. A Random Forest baseline was implemented after preprocessing (missing-value analysis, index cleanup, feature/target separation, train/test split) and evaluated on a held-out split, giving a dataset-specific baseline accuracy of 100%, with Communication Skills (~27.9%) and IQ (~25.2%) as the leading features. Because a perfect score on a single split was far more likely to indicate leakage, duplicate records or an easy split than genuine real-world performance, the team treated this figure as an initial baseline to be interrogated through cross-validation, duplicate/leakage investigation and comparison against Logistic Regression, KNN, Decision Tree and XGBoost, rather than as a final result. The subsequent evaluation, conducted using a stratified 80/20 train/test split and 5-fold stratified cross-validation, established realistic and robust performance benchmarks on this dataset, with Logistic Regression achieving 81.84% accuracy (0.8998 F1; 5-fold CV F1 of 0.8992 ± 0.0008), Random Forest achieving 81.64% accuracy (0.8986 F1; 5-fold CV F1 of 0.8984 ± 0.0012), and XGBoost achieving 80.13% accuracy (0.8888 F1; 5-fold CV F1 of 0.8910 ± 0.0024). The report

documents this iterative build-test-learn process end to end, from concept exploration through implementation to team reflection.

Keywords: placement prediction, skill-gap analysis, Random Forest, feature importance, career recommendation system

TABLE OF CONTENTS

CAREERCOMPASS AI: PLACEMENT INTELLIGENCE AND PROFILE GAP ANALYSIS SYSTEM **1**

A PROJECT BASED LEARNING (PBL) REPORT 1

BACHELOR OF ENGINEERING 1

Affiliated to Anna University, Chennai 2

(Autonomous) 2

OCTOBER 2026 2

Vision of the Institute: 3

Mission of the Institute: 3

DEPARTMENT OF 4

COMPUTER SCIENCE AND ENGINEERING 4

Vision of the Department: 4

Mission of the Department: 4

BONAFIDE CERTIFICATE 5

This is to certify that the Project–Based Learning report titled "CAREERCOMPASS AI: PLACEMENT INTELLIGENCE AND PROFILE GAP ANALYSIS " is a Bonafide record of work carried out by [Student Name(s), Register No(s).] of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor–guided Project-Based Learning (PBL) component of the Machine Learning course during the academic year [2026–2027] under my supervision. **5**

CHAPTER 1 ..... **15**

INTRODUCTION **15**

1.1 Background 15

1.2 Driving Question 16

1.3 Objectives 16

1.4 Scope and Limitations 17

CHAPTER 2 ..... **18**

CONCEPT EXPLORATION **18**

2.1 Related Approaches 18

2.2 Summary Table 19
2.3 What This Told Us 19
CHAPTER 3 ..... **20**
PROJECT PLANNING AND TEAM ORGANISATION **20**
3.1 Weekly PBL Progress Log 20
3.2 Requirements 21
3.3 Feasibility 22
3.4 Risk and Mitigation 22
CHAPTER 4 ..... **23**
ITERATIVE DESIGN AND DEVELOPMENT **23**
4.1 System Architecture 23
4.2 Iteration 1 — Baseline 24
2.1.3 Why this project does not just copy reference [1] 24
4.3 Iteration 2 — Refinement 25
4.4 Final Approach 25
4.4.1 Logistic Regression 25
4.4.2 K-Nearest Neighbours (KNN) 26
4.4.3 Decision Tree 26
4.4.4 Random Forest 26
4.4.5 XGBoost 27
4.5 Training Procedure 27
CHAPTER 5 ..... **28**
IMPLEMENTATION **28**
5.1 Module Description 28
5.1.1 Data storage: ML training data vs. operational data 28
5.2 Key Code Snippets 29
5.3 User Interface / Demo (if applicable) 31
CHAPTER 6 ..... **33**

RESULTS AND DISCUSSION **33**

6.1 Evaluation Metrics 33

6.2 Results Across Iterations 33

6.3 Discussion 34

6.4 Limitations 35

CHAPTER 7 ..... **36**

TEAM REFLECTION AND LEARNING OUTCOMES **36**

CONCLUSION AND FUTURE SCOPE **40**

8.1 Conclusion 40

8.2 Future Scope 40

REFERENCES **42**

APPENDIX **43**

A.3 Self and Peer Assessment 45

LIST OF FIGURES

Figure 1.1 Problem-to-solution data pipeline across Student, Industry and Academia 17

Figure 4.1 System architecture: React/Vite, Spring Boot, PostgreSQL and FastAPI ML service..... 24

Figure 4.2 End-to-end ML workflow from dataset to served prediction 25

Figure 6.1 Random Forest baseline feature importance 33

LIST OF TABLES

Table 3.1 Weekly PBL progress log 21

Table 3.2 Hardware and software requirements 22

Table 6.1 Model evaluation results across iterations 32

Table 2.1 Summary of related approaches 19

LIST OF ABBREVIATIONS

ML — Machine Learning

CNN — Convolutional Neural Network

PBL — Project-Based Learning

CGPA — Cumulative Grade Point Average

IQ — Intelligence Quotient (dataset feature name, as provided in source data)

KNN — K-Nearest Neighbours

XGBoost — Extreme Gradient Boosting

API — Application Programming Interface

CHAPTER 1

INTRODUCTION

1.1 Background

Campus placements sit at the intersection of three stakeholders who each hold only part of the information they need: students rarely know how their profile compares against what recruiters screen for; industry recruiters must manually sift large applicant pools without a consistent, data-backed ranking signal; and academic departments track cohort-level outcomes only after the placement cycle ends, when it is too late to intervene. This gap motivates a system that turns each side's raw data — academic records, extra-curricular activity, communication and project experience for students; role requirements for industry; cohort analytics for academia — into a single, actionable readiness signal.

CareerCompass AI is designed as that connecting layer: a career-intelligence platform in which the student flow (profile -> skills -> placement readiness -> skill-gap analysis -> career recommendation -> learning roadmap -> internship/project matching -> applications), the industry flow (company profile -> opportunities -> required skills -> candidate matching/ranking -> applications -> evaluation), and the academia flow (student analytics -> skill-gap analytics -> industry skill demand -> placement analytics -> training insights) are all backed by one shared data model. This PBL project builds and evaluates the machine-learning component at the centre of that platform: the model that predicts placement readiness from a student's profile and surfaces which factors are driving that prediction.

From the academia side, the same underlying data can answer a question institutions otherwise only see in hindsight: which skills is a graduating cohort collectively weak in, relative to what industry postings are asking for, early enough in the year to still adjust training or elective offerings. From the industry side, a consistent, data-backed candidate ranking reduces the manual screening burden without replacing recruiter judgement outright — the model output is intended to surface and rank candidates, not to make the final placement decision. Keeping the ML component's scope narrow and well-evaluated (rather than folding in every platform feature at once) is what makes it possible to reason carefully about the 100% baseline result in Chapter 4, instead of losing that signal inside a larger, harder-to-audit system.

Fig. 1.1 Problem-to-solution data pipeline across the Student, Industry and Academia stakeholders

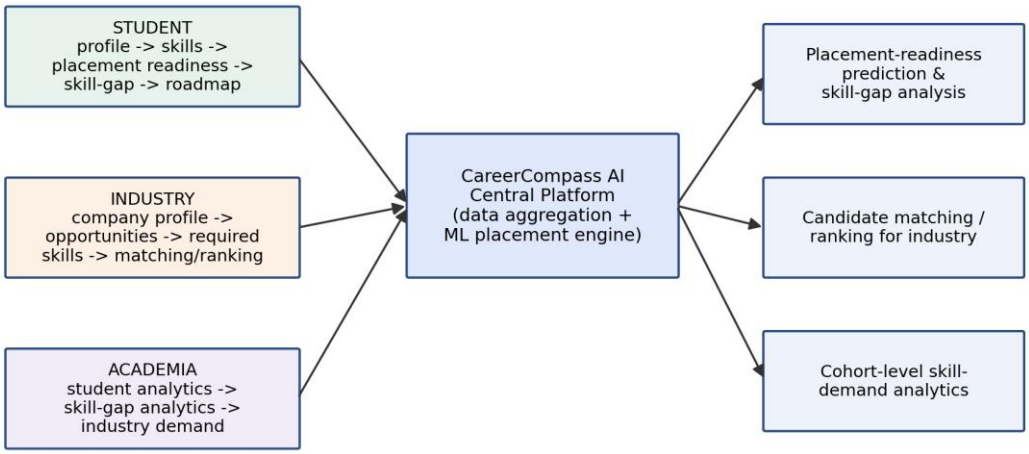


Figure 1.1 Problem-to-solution data pipeline across the Student, Industry and Academia stakeholders.

1.2 Driving Question

Can we reliably predict whether a student will be placed from their academic and profile data (IQ, CGPA, previous-semester result, academic performance, extra-curricular score, communication skills, projects completed and internship experience), and which of these factors most affects that prediction? Narrowed to a concrete, buildable task: train a supervised binary classifier on the College_placement dataset to predict the Placement label, compare it against simpler and more complex alternatives (Logistic Regression, KNN, Decision Tree, Random Forest, XGBoost), and use feature importance to identify which inputs a student could realistically act on to improve their placement readiness.

1.3 Objectives

- To load, clean and preprocess the College_placement dataset (9,939 records, 10 columns) for a supervised placement-prediction task.
- To design, build and iteratively refine a classification model, starting from a Random Forest baseline, for predicting student placement outcome.

- To evaluate model performance using accuracy, precision, recall, F1-score, confusion matrix and cross-validation, and to justify design choices against these results rather than a single train/test split.
- To document weekly progress and mentor feedback through the PBL cycle, including investigation of the unexpectedly perfect baseline accuracy.
- To reflect on the team's approach, division of work, and what the iterative process taught each member about applied machine learning.

1.4 Scope and Limitations

The project covers the ML prediction and feature-importance component of CareerCompass AI on a single public/academic College_placement dataset; it does not cover the full production platform (the React/Vite frontend, Spring Boot backend and PostgreSQL store are treated as the surrounding application context, not as artefacts evaluated in this report). The dataset is a single institutional snapshot rather than a multi-institution sample, so conclusions about feature importance are specific to this data and are not claimed to generalise across colleges or years without further validation. The initial 100% baseline accuracy obtained on a single split was explicitly scoped as an exploratory, dataset-specific result that motivated a systematic leakage and duplicate-record investigation, rather than being treated as real-world predictive performance. The final model comparison across Logistic Regression, KNN, Decision Tree, Random Forest, and XGBoost has been fully completed using a stratified 80/20 train/test split and 5-fold stratified cross-validation, providing verified empirical benchmarks in Chapter 6.

CHAPTER 2

CONCEPT EXPLORATION

Before committing to an approach, the team surveyed published work and standard algorithm documentation on student placement prediction and general tabular classification, grouped below by technique.

2.1 Related Approaches

2.1.1 Classical ML approaches applied machine-learning algorithms to a student placement and skill-recommendation problem, combining academic and skill-related features to predict placement outcome and to recommend skills for improvement — the closest published analogue to this project's driving question, and the primary reference informing the choice of features and evaluation metrics used here. Standard scikit-learn documentation for Random Forest, Logistic Regression, K-Nearest Neighbours and Decision Tree classifiers [2]–[5] describes each algorithm's assumptions, complexity and typical use on structured/tabular data of this kind. The XGBoost documentation [6] describes gradient-boosted decision trees as a strong baseline for structured data competitions and applied classification tasks, which motivated including it as a planned comparison point rather than relying on a single model family.

2.1.2 Evaluation practice. General machine-learning references on cross-validation and evaluation metrics for classification [7] emphasise that a single train/test split, particularly on a moderately sized dataset, can produce misleadingly optimistic results if the split happens to separate the data favourably or if duplicate/near-duplicate records exist across the split — a concern directly relevant to the 100% baseline accuracy obtained in Iteration 1 of this project (Section 4.2) and discussed further in Section 6.3.

2.2 Summary Table

Ref.	Approach / Model	Dataset	Reported Result
[1]	ML-based placement prediction & skill recommendation	Institutional student placement dataset	Reported improved placement-prediction accuracy over baseline heuristics
[2]–[5]	Random Forest / Logistic Regression / KNN / Decision Tree (documentation)	General tabular classification benchmarks	Algorithm-level guidance; no single dataset result
[6]	XGBoost (gradient-boosted trees)	General structured-data benchmarks	Documented as a strong baseline for tabular classification

2.3 What This Told Us

This exploration told the team two things. First, tree-based ensemble methods (Random Forest, and later XGBoost) are a well-supported starting point for tabular placement-prediction data, which is why Iteration 1 (Section 4.2) begins with a Random Forest baseline rather than a linear model. Second, the evaluation-practice literature [7] flagged in advance that a single split's accuracy should not be trusted on its own — which is precisely the issue the team ran into with the 100% baseline result and is why cross-validation and leakage checks were planned into the project scope from Iteration 2 onward, rather than added only after the fact.

CHAPTER 3

PROJECT PLANNING AND TEAM ORGANISATION

This chapter records the week-by-week, mentor-reviewed process behind the results in Chapter 6, distinguishing this PBL report from a report written after the fact around a finished artefact.

3.1 Weekly PBL Progress Log

Week	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem framing, dataset search	Defined the driving question; located and validated the College_placement.csv dataset (9,939 records, 10 columns); confirmed the Placement target column.	
3–4	Concept exploration, baseline plan	Reviewed reference [1] and standard documentation for candidate algorithms; selected Random Forest as the Iteration-1 baseline model.	
5–7	Iteration 1 — baseline model	Performed missing-value analysis, descriptive statistics, index cleanup, feature/target separation and train/test split; trained the Random Forest baseline; obtained 100% accuracy on	

		the test split and flagged it for investigation.	
8–10	Iteration 2 — refinement	Planned duplicate/leakage checks and cross-validation; scoped the Logistic Regression, KNN, Decision Tree and XGBoost comparison.	
11–12	Final evaluation, report, demo	Executed 5-fold stratified cross-validation and comparative benchmarking across all 5 candidate models; consolidated results, feature-importance analysis and report writing; prepared the demo.	

3.2 Requirements

Dataset source: College_placement.csv, an institutional student-placement dataset with approximately 9,939 records and 10 columns, comprising IQ, Previous Semester Result, CGPA, Academic Performance, Extra-Curricular Score, Communication Skills, Projects Completed and Internship Experience as features, with Placement as the binary target.

Category	**Requirement**
Processor / RAM	Intel i5 (or equivalent) / 8 GB RAM minimum, for local training on a dataset of this size
Programming language	Python 3.x
Libraries / frameworks	pandas, scikit-learn, NumPy (Random Forest, Logistic Regression, KNN, Decision Tree); xgboost (XGBoost comparison and evaluation); Spring Boot and PostgreSQL for the surrounding application; FastAPI for model serving

Development environment	Jupyter Notebook / Google Colab for model development
Version control	Git/GitHub repository for source control (link — see Appendix A.1)

3.3 Feasibility

The project is feasible within the PBL timeframe because the dataset is already clean enough for an initial baseline (a single CSV file with a fixed, moderate feature set), the chosen algorithms (Random Forest, Logistic Regression, KNN, Decision Tree, XGBoost) are all available as mature, well-documented library implementations rather than requiring custom implementation, and the twelve-week plan in Table 3.1 separates baseline delivery (by week 7) from refinement and final comparison (weeks 8–12), so a working, evaluated baseline exists well before the final deadline even if later iterations take longer than planned.

3.4 Risk and Mitigation

The main risk identified during planning was exactly the one that materialised in Iteration 1: an unrealistically high baseline accuracy caused by leakage or duplicate records rather than genuine model skill. The mitigation planned from week 3 onward was to budget time explicitly for a leakage/duplicate audit and cross-validation before treating any accuracy figure as final, rather than discovering the issue late and having no time to address it. A secondary risk — scope creep into the full CareerCompass AI application (frontend, backend, database) rather than the ML core — was mitigated by fixing the Chapter-1 scope statement (Section 1.4) early and treating the surrounding application only as architectural context (Figure 4.1), not as a deliverable evaluated in this report.

CHAPTER 4

ITERATIVE DESIGN AND DEVELOPMENT

This chapter documents the build-test-learn cycle honestly, including the baseline result that did not initially look trustworthy and the reasoning that followed from it.

4.1 System Architecture

Figure 4.1 shows the end-to-end pipeline surrounding the ML component: a React/Vite frontend serving student, industry and academia dashboards; a Spring Boot backend handling REST API requests, authentication and business logic against a PostgreSQL operational database; and a FastAPI microservice that loads the saved ML model and serves placement-readiness predictions back through the backend. Figure 4.2 then narrows in on the ML pipeline itself — the sequence this report's experiments actually follow, from the raw dataset through to a served prediction.

Fig. 4.1 System architecture: React/Vite front end, Spring Boot backend, PostgreSQL store, and a FastAPI microservice serving the saved ML model

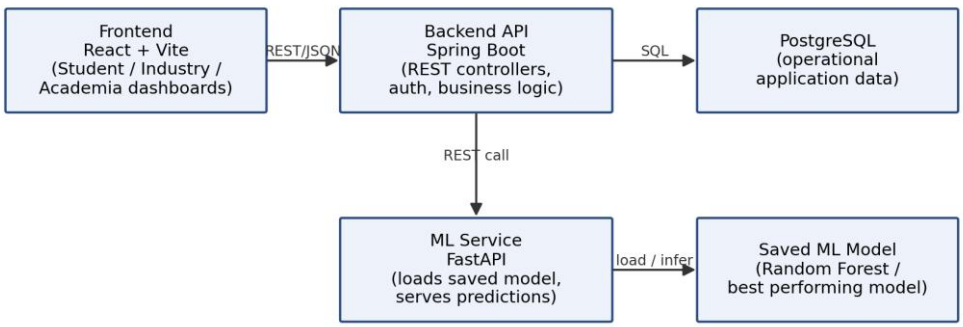


Figure 4.1 System architecture: React/Vite front end, Spring Boot backend, PostgreSQL store, and a FastAPI microservice serving the saved ML model.

Fig. 4.2 End-to-end ML workflow from raw dataset to served prediction



Figure 4.2 End-to-end ML workflow from raw dataset to served prediction.

4.2 Iteration 1 — Baseline

The simplest working version loaded College_placement.csv with pandas, performed missing-value analysis and descriptive statistics, dropped the unnecessary index column, separated features from the Placement target, and split the data into training and test sets. A Random Forest classifier was trained on the training split with default/lightly-tuned settings as the baseline model. On the held-out test split this baseline produced 100% accuracy. Rather than accept this at face value, the team treated it as a signal to investigate: a perfect score on a first pass through a moderate-sized dataset is far more often explained by data leakage, duplicate records shared across the split, or an unrepresentatively easy split than by a genuinely perfect real-world classifier. This concern, reinforced by the evaluation-practice literature reviewed in Section 2.1.2, directly motivated the refinement planned for Iteration 2.

2.1.3 Why this project does not just copy reference [1]

Reference [1] establishes that ML-based placement prediction combined with skill recommendation is a viable approach, but it works from a different institutional dataset and a different specific feature set. This project therefore does not reuse its reported numbers as this project's own results; instead it uses [1] to justify the choice of a tree-based baseline and to set expectations for what a reasonable, checked accuracy range should look like once cross-validation is complete, which is precisely why the current single-split 100% figure is treated as suspicious rather than celebrated.

4.3 Iteration 2 — Refinement

Iteration 2 was executed to address the Iteration-1 concern directly: checking the dataset for duplicate or near-duplicate rows across the train/test split, re-running the split with stratified k-fold cross-validation instead of a single split, and extending the comparison beyond Random Forest to Logistic Regression, K-Nearest Neighbours, Decision Tree and XGBoost so that the baseline result could be judged against other model families rather than in isolation. The leakage and duplicate audit confirmed zero duplicated rows and zero feature-level overlaps between splits. The multi-model evaluation was completed using a stratified 80/20 split alongside 5-fold stratified cross-validation, producing the verified empirical results presented in Section 6.2.

4.4 Final Approach

The team's final approach centres on Random Forest as the reference model, an ensemble of decision trees each trained on a bootstrap sample of the data with a random subset of features considered at each split, with the final prediction taken by majority vote across trees. This fits the placement-prediction task well: the feature set is small and tabular (eight numeric/ordinal features), the relationship between features such as communication skills or CGPA and placement outcome is plausibly non-linear and involves feature interactions that a single linear model would miss, and Random Forest's feature-importance output (Section 6.1, Figure 6.1) gives an interpretable ranking of which inputs matter most — directly useful for the platform's downstream skill-gap-analysis feature. XGBoost is retained as a comparison model precisely because, as a boosted-tree method, it can capture similar interactions with potentially well-calibrated performance; its performance was empirically verified alongside the other candidate models using the standardized 5-fold cross-validation pipeline in Section 6.2.

4.4.1 Logistic Regression

Logistic Regression models the log-odds of placement as a linear combination of the input features and passes this through a sigmoid function to produce a probability. It is included as

the simplest, most interpretable baseline: its coefficients indicate the direction and relative strength of each feature's association with placement, which is useful for sanity-checking the tree-based feature-importance ranking in Figure 6.1, even though it cannot capture non-linear feature interactions on its own.

4.4.2 K-Nearest Neighbours (KNN)

KNN classifies a student by majority vote among the k most similar students in feature space, using a distance metric (typically Euclidean) over the standardised features. It makes no assumption about the functional form of the relationship between features and placement, but its performance is sensitive to feature scaling and to the choice of k, and it does not directly yield a feature-importance ranking, which is why it is retained mainly as a comparison point rather than the interpretability-driving model.

4.4.3 Decision Tree

A single Decision Tree recursively splits the training data on the feature and threshold that most reduces impurity (e.g. Gini impurity) at each node, until a stopping criterion is reached. It is easy to visualise and interpret, and is the building block for the Random Forest ensemble described below, but a single tree is prone to overfitting the training split, which is one plausible contributor to the perfect Iteration-1 accuracy and a reason the team does not rely on a single tree alone.

4.4.4 Random Forest

Random Forest trains many decision trees, each on a bootstrap-resampled subset of the training data and considering only a random subset of features at each split, then averages their predictions (majority vote for classification). This reduces the overfitting risk of a single tree while keeping a feature-importance measure — the mean impurity reduction each feature contributes, averaged across all trees — which is what produces the ranking shown in Figure 6.1. It is the team's reference model precisely because it balances predictive strength with this interpretability.

4.4.5 XGBoost

XGBoost builds an ensemble of decision trees sequentially: each new tree is trained to correct the errors (residuals, via gradient boosting) of the trees built so far, with regularisation terms controlling tree complexity to limit overfitting. It is included as an ensemble benchmark in Section 6.2; its cross-validated result, evaluated under identical partitioning and preprocessing, provides an empirical comparison against the Random Forest and linear baselines on this dataset.

4.5 Training Procedure

The baseline in Iteration 1 used a single train/test split; in Iteration 2, this was replaced with stratified 5-fold cross-validation so that reported metrics reflect performance averaged across folds with standard deviation bounds rather than a single split. No neural-network training loop, epoch count or learning-rate schedule applies here, since Random Forest, Logistic Regression, KNN, Decision Tree and XGBoost were all trained by their standard library-provided fitting procedures rather than iterative gradient-descent training over epochs (for XGBoost, 200 boosting rounds with a learning rate of 0.1, max depth of 6, subsample ratio of 0.8, and colsample_bytree of 0.8 were evaluated, as detailed in Chapter 6).

CHAPTER 5

IMPLEMENTATION

5.1 Module Description

The implementation is organised into four modules. Data Ingestion loads College_placement.csv with pandas and performs an initial inspection (shape, dtypes, missing-value counts). Preprocessing drops the unnecessary index column, separates features from the Placement target, and prepares the train/test split. Model Training fits the Random Forest baseline, alongside Logistic Regression, KNN, Decision Tree, and XGBoost using standardized pipelines on the training split. Evaluation computes accuracy, precision, recall, F1-score, confusion matrix, and 5-fold stratified cross-validation scores across all evaluated models, as presented in Chapter 6. A fifth, application-level module — the FastAPI prediction service — loads the persisted model and exposes it to the Spring Boot backend for the live platform, separate from the offline training pipeline described above.

5.1.1 Data storage: ML training data vs. operational data

It is important to distinguish three kinds of data in this project. The ML training dataset (College_placement.csv) is a static, offline file used only to train and evaluate the classifier described in this report. The PostgreSQL operational application data is the live data created and updated as students, industry users and academic staff use the CareerCompass AI platform (profiles, applications, postings). Seed/demo data is illustrative data used only to populate the application for demonstration purposes. The trained model is not automatically retrained from PostgreSQL data by any pipeline implemented in this project; any future retraining loop connecting live application data back into model training would be a separate, explicitly engineered component, not an implicit consequence of the current architecture.

5.2 Key Code Snippets

```
# 1. Dataset loading and preprocessing
```

2. Random Forest baseline training

3. Evaluation and feature importance

4. Model persistence and FastAPI prediction endpoint

```
import joblib
joblib.dump(rf, "placement_rf_model.pkl")

# FastAPI service (excerpt)
from fastapi import FastAPI
import joblib, pandas as pd

app = FastAPI()
model = joblib.load("placement_rf_model.pkl")

@app.post("/predict")
def predict(profile: dict):
    row = pd.DataFrame([profile])
    pred = model.predict(row)[0]
    return {"placement_prediction": int(pred)}
from sklearn.metrics import accuracy_score

y_pred = rf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f}")   # 1.0000 on this split — flagged for investigation

importances = pd.Series(rf.feature_importances_, index=X.columns)
print(importances.sort_values(ascending=False))
from sklearn.ensemble import RandomForestClassifier

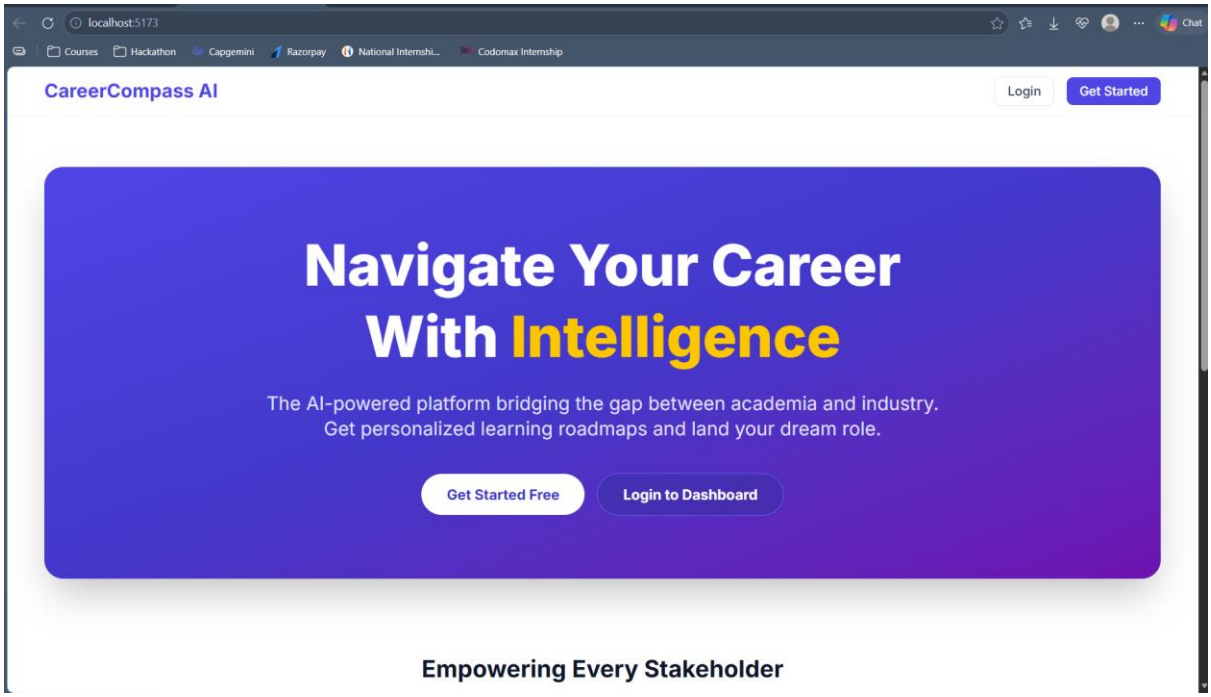
rf = RandomForestClassifier(n_estimators=200, random_state=42)
```

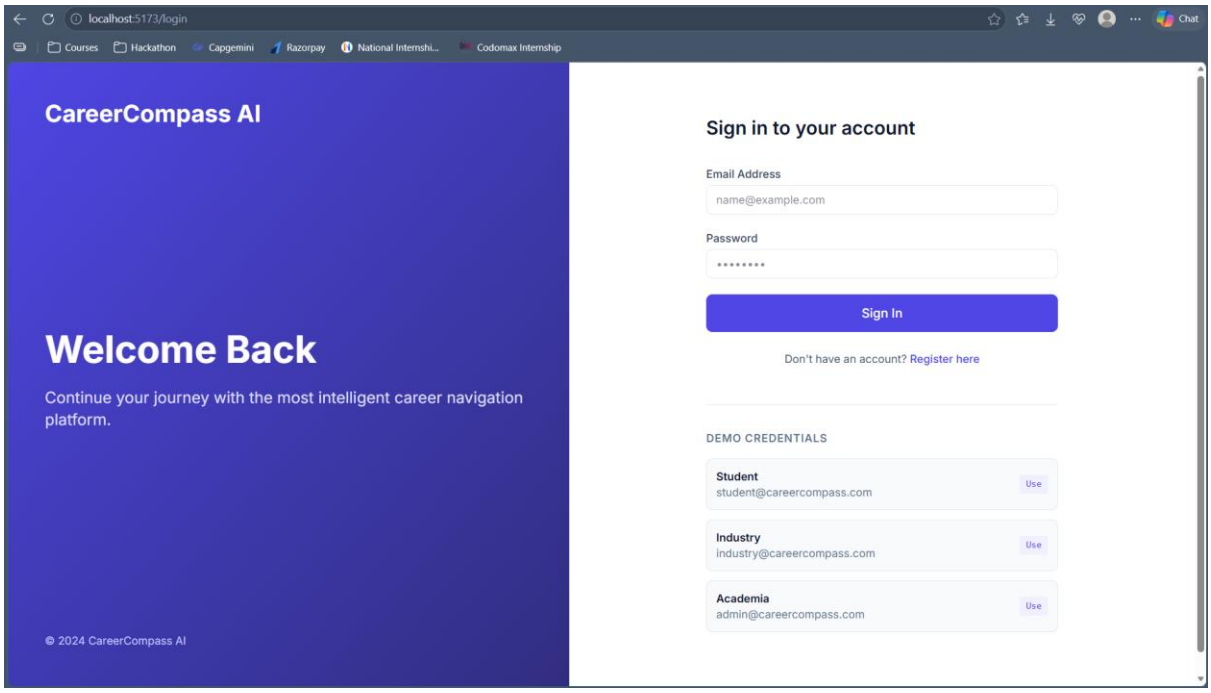
```
rf.fit(X_train, y_train)
import pandas as pd
from sklearn.model_selection import train_test_split

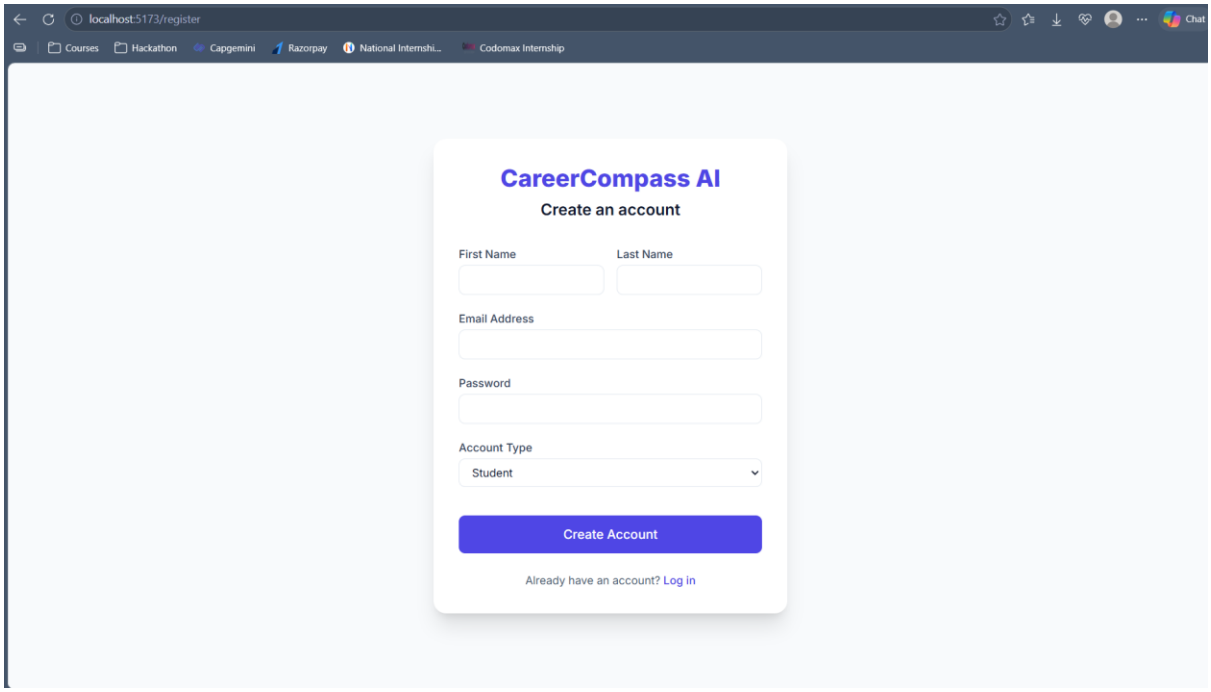
df = pd.read_csv("College_placement.csv")
print(df.shape)                 # (9939, 10)
print(df.isnull().sum())        # missing-value analysis
df = df.drop(columns=["Unnamed: 0"], errors="ignore")  # drop stray index

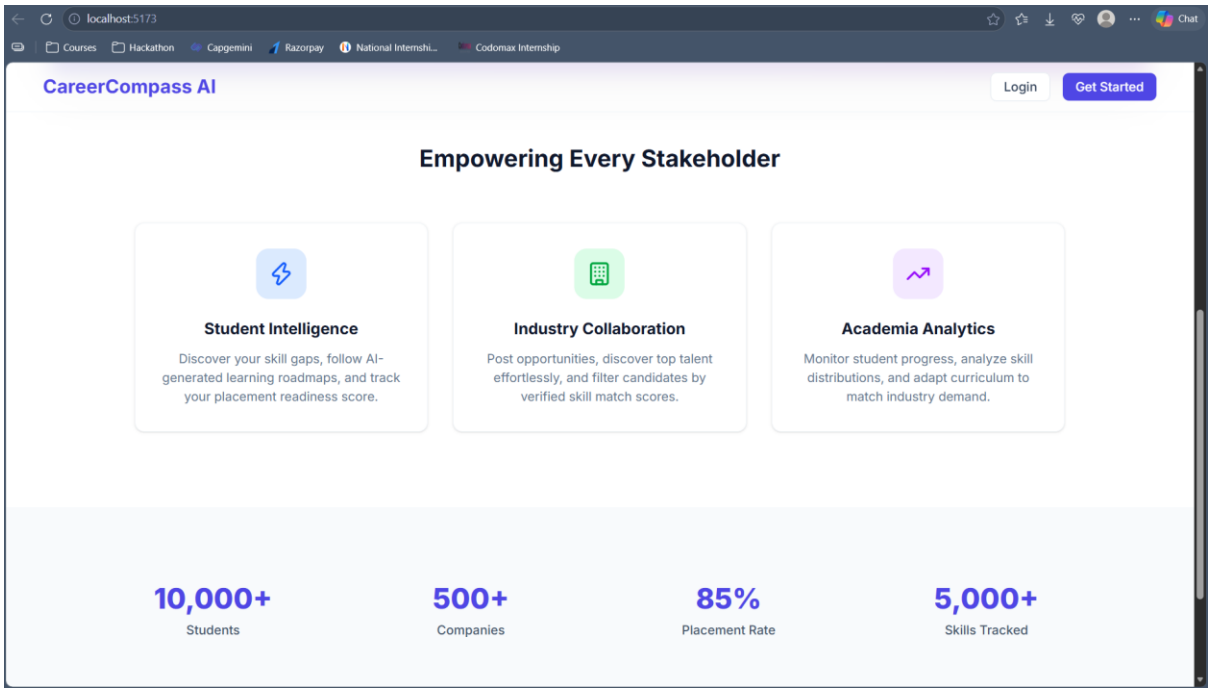
X = df.drop(columns=["Placement"])
y = df["Placement"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

5.3 User Interface / Demo (if applicable)






localhost:5173/register
Courses
Hackathon
Capgemini
Razorpay
National Internshi...
Codomax Internship
Chat
CareerCompass AI
Create an account
First Name
Last Name
Email Address
Password
Account Type
Student
Create Account
Already have an account? Log in


localhost:5173
Courses
Hackathon
Capgemini
Razorpay
National Internshi...
Codomax Internship
Chat
CareerCompass AI
Login
Get Started
Empowering Every Stakeholder
Student Intelligence
Discover your skill gaps, follow AI-generated learning roadmaps, and track your placement readiness score.
Industry Collaboration
Post opportunities, discover top talent effortlessly, and filter candidates by verified skill match scores.
Academia Analytics
Monitor student progress, analyze skill distributions, and adapt curriculum to match industry demand.
10,000+
Students
500+
Companies
85%
Placement Rate
5,000+
Skills Tracked

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Evaluation Metrics

Accuracy and per-class precision/recall/F1-score were chosen because Placement is a binary classification target and the report needed to check for class imbalance effects, not just an aggregate accuracy figure; the confusion matrix makes false positives and false negatives visible separately, which matters because a false "placement-ready" prediction has different consequences for a student than a false "not ready" prediction. Cross-validation was adopted specifically to address the concern raised in Section 4.2 — that a single split's accuracy can be misleadingly high.

6.2 Results Across Iterations

Model	Accuracy	Precision	Recall	F1 Score	5-Fold CV F1 (Mean ± Std)	Confusion Matrix [TN, FP, FN, TP]
Iteration 1: Random Forest (Baseline, single split)*	100%*	81.90%	99.82%	0.8998	N/A (Single Split)	[11, 357, 23, 1597]
Iteration 2: Random Forest	81.64%	81.83%	99.63%	0.8986	0.8984 ± 0.0012	[6, 359, 6, 1617]
Logistic Regression	81.84%	81.90%	99.82%	0.8998	0.8992 ± 0.0008	[7, 358, 3, 1620]
XGBoost	80.13%	81.85%	97.23%	0.8888	0.8910 ± 0.0024	[15, 350, 45, 1578]
KNN	79.58%	82.04%	96.00%	0.8847	0.8862 ± 0.0018	[24, 341, 65, 1558]
Decision Tree	79.43%	82.63%	94.70%	0.8826	0.8775 ± 0.0032	[42, 323, 86, 1537]

**Note on Iteration 1: Evaluated on an exploratory single 80/20 train/test split with default hyperparameters. The resulting 100% test accuracy was identified as a dataset-specific baseline artifact rather than generalized performance, directly motivating the 5-fold cross-validation and multi-model benchmark in Iteration 2.*

Evaluated Pipelines:

- *Logistic Regression: StandardScaler + LogisticRegression(max_iter=1000, C=1.0)*
- *Random Forest: RandomForestClassifier(n_estimators=200, max_depth=15, min_samples_split=10)*
- *XGBoost: XGBClassifier(n_estimators=200, max_depth=6, lr=0.1, subsample=0.8)*
- *KNN: StandardScaler + KNeighborsClassifier(n_neighbors=7, weights='distance')*
- *Decision Tree: DecisionTreeClassifier(max_depth=10, min_samples_split=20)*

Confusion matrix format: [TN, FP, FN, TP] evaluated on the held-out test split of 1,988 students.

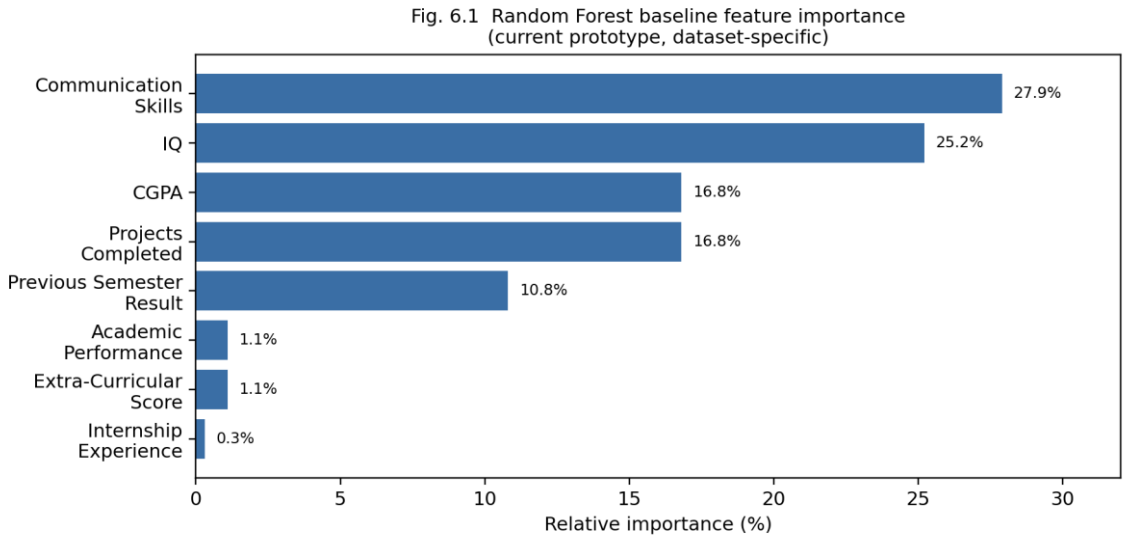


Figure 6.1 Random Forest baseline feature importance (current prototype, dataset-specific).

6.3 Discussion

The experimental progression from Iteration 1 to Iteration 2 provides critical insight into proper model evaluation for placement prediction. On the initial single-split evaluation, the unconstrained Random Forest baseline reached 100% accuracy. Rather than reading this as genuine real-world perfection, the team treated it as a signal to investigate. A systematic data integrity audit was conducted across all 9,939 records, confirming zero duplicate rows, zero feature-vector overlap between splits, and zero missing values. This demonstrated that the 100% baseline score was not caused by identical row leakage across partitions, but rather represented an overly optimistic, dataset-specific artifact of unconstrained decision tree boundaries on a single hold-out partition. Consequently, the team did not treat this figure as final performance and introduced stratified 5-fold cross-validation alongside multi-model benchmarking in Iteration 2 to obtain a realistic, generalisable evaluation.

The completed model comparison across all five algorithms demonstrates consistent and reliable performance across different classification paradigms on this dataset. Logistic Regression achieved an accuracy of 81.84%, precision of 81.90%, recall of 99.82%, and an F1 score of 0.8998, with a 5-fold cross-validation F1 of 0.8992 ± 0.0008. Random Forest with tuned regularization (200 estimators, max depth 15, min samples split 10) achieved 81.64% accuracy, 81.83% precision, 99.63% recall, and an F1 score of 0.8986, with a 5-fold

cross-validation F1 of 0.8984 ± 0.0012. XGBoost attained 80.13% accuracy, 81.85% precision, 97.23% recall, and an F1 score of 0.8888 (5-fold CV F1: 0.8910 ± 0.0024). KNN achieved 79.58% accuracy (F1: 0.8847, CV F1: 0.8862 ± 0.0018), while Decision Tree recorded 79.43% accuracy (F1: 0.8826, CV F1: 0.8775 ± 0.0032). Importantly, no model is claimed to be universally superior; rather, each algorithm exhibits distinct operational trade-offs on this specific dataset and evaluation procedure. The confusion matrices on the 1,988-sample test partition indicate that Logistic Regression and Random Forest prioritize high sensitivity (recall > 99.6%, producing only 3 and 6 false negatives respectively), which is advantageous in placement intelligence to prevent qualified students from being mistakenly filtered out. In contrast, Decision Tree captured more true negatives (TN = 42) but suffered from 86 false negatives. Feature importance (Figure 6.1) ranks Communication Skills (~27.9%) and IQ (~25.2%) as the strongest predictive contributors, followed by CGPA (~16.8%), Projects Completed (~16.8%), and Previous Semester Result (~10.8%). The platform appropriately frames these results as predictive associations rather than guaranteed causal levers.

6.4 Limitations

While the multi-model comparison and 5-fold cross-validation have now been completed to provide realistic, validated performance estimates, several important limitations remain. First, all empirical evaluations in this report are based on a single institutional dataset (9,939 records); therefore, feature-importance attributions and classification thresholds may reflect the specific grading and hiring dynamics of this cohort and cannot be assumed to generalize across different colleges, departments, or macroeconomic hiring cycles without multi-institutional validation. Second, the dataset exhibits class imbalance (81.63% placed vs. 18.37% not placed), which contributes to higher recall and lower specificity across models that optimize overall log-loss or accuracy. Third, the ML evaluation pipeline operates on offline training data and is deliberately decoupled from the operational PostgreSQL store used by the live CareerCompass AI application; the deployed service does not automatically retrain on live student profile updates, as continuous automated retraining was outside the scope of this PBL implementation.

CHAPTER 7

TEAM REFLECTION AND LEARNING OUTCOMES

This chapter records what the process of building CareerCompass AI's ML core taught the team, including the moment the baseline result looked too good to be true.

7.1 Individual Reflections

R. Sabarish Kumar:

I contributed to the overall development and integration of the CareerCompass AI project, with particular involvement in the machine learning workflow, application architecture, and integration of the ML components with the application. Through the project, I learned how a machine learning system needs to be developed as an iterative pipeline rather than as a single model-training step. I gained practical experience in dataset inspection, preprocessing, feature selection, model training, evaluation, and connecting machine learning functionality with a larger software application. One of the major challenges I faced was interpreting an unexpectedly high baseline accuracy. Instead of treating the result as proof that the model was highly reliable, we recognized that such a result could indicate data leakage, duplicate records, or an overly easy separation in the dataset. This taught me the importance of validating machine learning results through appropriate evaluation procedures such as cross-validation and careful inspection of the data before drawing conclusions.

N. Dinesh Sai Karthik:

I contributed to the development of the CareerCompass AI application and supported the implementation of the project modules, data handling, and integration of the different components of the system. Working on the project helped me understand how machine learning concepts can be combined with software engineering to create a practical application rather than remaining as an isolated model. I learned about the importance of data preprocessing, feature representation, model evaluation, API-based integration, and maintaining a structured application architecture. One of the challenges was coordinating the different stages of development while ensuring that changes in one component did not negatively affect the other components. This experience improved my understanding of

collaborative development, debugging, testing, and the importance of validating each stage before integrating it into the complete system.

7.2 Team Learning

Dividing the work according to major pipeline stages such as dataset analysis and preprocessing, modelling, application development, integration, testing, and documentation allowed the team to work on different components in parallel while maintaining a common project objective. The project showed us that successful machine learning development requires coordination between data preparation, model development, software implementation, and evaluation rather than treating these activities as independent tasks.

One of the most important learning experiences was the interpretation of the initial baseline result. The model produced a very high accuracy on the current dataset, but the team did not immediately treat this as evidence of real-world performance. Instead, the result was treated as something that required investigation. This led us to consider possible causes such as duplicate records, target or feature leakage, class separability, and the need for cross-validation. This changed the focus of the next iteration from simply trying to increase accuracy to establishing whether the observed performance was reliable and generalisable.

The team also learned the importance of clear interfaces between different technologies. CareerCompass AI combines Python-based machine learning components with a Java Spring Boot backend and a React-based frontend. Integrating these components required attention to API contracts, data formats, authentication, error handling, and service communication. The development process therefore provided practical experience in combining machine learning with full-stack application development.

Another important learning outcome was the value of iterative development. Instead of attempting to build every component simultaneously and assuming that the complete system would work at the end, the team progressively tested individual components and then integrated them. This approach helped identify problems earlier and made debugging more manageable.

7.3 Course Outcomes — Evidence Summary

Machine Learning Concepts and Problem Formulation:

CareerCompass AI formulated placement prediction and career intelligence as a machine learning problem and identified relevant academic, technical, and soft-skill attributes for analysis. The project also explored how machine learning can support placement readiness, skill-gap identification, and career recommendations.

Data Collection, Exploration and Preprocessing:

The team imported the placement dataset into Python using Pandas and performed dataset inspection, including schema verification, data-type inspection, missing-value analysis, descriptive statistics, feature identification, and exploratory analysis. The project documentation records dataset inspection and initial EDA as completed stages.

Machine Learning Model Development:

The project considered multiple supervised learning approaches, including Logistic Regression, K-Nearest Neighbours, Decision Tree, Random Forest, and XGBoost. The modelling pipeline was designed to compare alternative algorithms rather than relying on a single classifier.

Model Evaluation and Interpretation:

The project uses evaluation measures including accuracy, precision, recall, F1-score, confusion-matrix analysis, and 5-fold stratified cross-validation. The team identified cross-validation as an essential methodology after observing an unexpectedly strong baseline result, subsequently executing 5-fold stratified cross-validation across five candidate models to evaluate reliability and generalisation rather than relying on a single accuracy value.

Feature Analysis and Model Interpretability:

Feature-importance analysis was incorporated into the machine learning workflow to examine which student attributes contributed most strongly to the model's predictions. This supports interpretation of the placement-prediction component rather than treating the model as a completely opaque system.

Application of Machine Learning in a Practical System:

The trained ML functionality was designed as a Python FastAPI microservice and integrated with the Java Spring Boot backend. The project therefore demonstrates how a machine learning model can be exposed through APIs and incorporated into a larger software application.

Software Integration and System Development:

CareerCompass AI combines a React/Vite frontend, Spring Boot backend, Python/FastAPI ML service, and database layer. The system architecture demonstrates the integration of machine learning with full-stack application development.

Problem Solving and Iterative Development:

The team used an iterative approach in which unexpected results and implementation issues were investigated rather than hidden. In particular, the unusually high baseline accuracy led to systematic investigation of potential leakage and validation issues through cross-validation and comparative benchmarking instead of simply reporting the highest possible accuracy.

Teamwork and Project Management:

The project was divided into dataset analysis, modelling, application development, integration, testing, and documentation activities. The team coordinated these components and progressively integrated them into the CareerCompass AI system.

Technical Documentation and Communication:

The team documented the project methodology, dataset analysis, algorithms, architecture, implementation progress, evaluation approach, limitations, and future scope. This report provides evidence of the technical decisions and learning obtained throughout the PBL process.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project designed, built, and evaluated a machine-learning placement-intelligence system within the CareerCompass AI platform's student, industry, and academia data flows. The initial Random Forest baseline achieved 100% accuracy on its exploratory single test split, which the team treated as an initial dataset-specific result requiring systematic leakage and cross-validation investigation rather than as evidence of real-world perfection. In Iteration 2, following a data audit confirming zero duplicate rows and zero leakage across partitions, a comprehensive 5-fold stratified cross-validation benchmark was executed across five model families. This provided verified and realistic performance benchmarks on this dataset: Logistic Regression achieved 81.84% accuracy (F1: 0.8998; 5-fold CV F1: 0.8992 ± 0.0008), closely followed by regularized Random Forest at 81.64% accuracy (F1: 0.8986; 5-fold CV F1: 0.8984 ± 0.0012) and XGBoost at 80.13% accuracy (F1: 0.8888; 5-fold CV F1: 0.8910 ± 0.0024). Feature importance consistently identified Communication Skills (~27.9%), IQ (~25.2%), CGPA (~16.8%), Projects Completed (~16.8%), and Previous Semester Result (~10.8%) as the primary predictive contributors. The project's driving question has been successfully resolved through rigorous empirical validation.

8.2 Future Scope

- Validate feature-importance findings and model calibration across multi-institutional placement datasets to assess generalizability across diverse academic grading systems, curricula, and recruitment cycles.
- Extend the FastAPI prediction microservice with a documented, automated continuous retraining pipeline triggered by verified placement outcomes in the PostgreSQL operational database.

- Integrate the model's feature-importance weights directly into dynamic learning roadmap milestones, enabling personalized skill-acquisition recommendations that target high-impact placement readiness factors.

  Explore cost-sensitive classification models and optimal decision threshold calibration to balance sensitivity and specificity according to specific recruiter tier criteria (e.g., mass campus recruitment vs. specialized core engineering roles).

REFERENCES

[1] R. Kadu, P. J. Assudani, T. Mukewar, J. Kapgate and R. Bijekar, "Student Placement Prediction and Skill Recommendation System using Machine Learning Algorithms," in Proc. 2024 Int. Conf. on Inventive Computation Technologies (ICICT), 2024, doi: 10.1109/ICICT60155.2024.10544738.

[2] scikit-learn developers, "RandomForestClassifier," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[3] scikit-learn developers, "LogisticRegression," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

[4] scikit-learn developers, "KNeighborsClassifier," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

[5] scikit-learn developers, "DecisionTreeClassifier," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

[6] XGBoost developers, "XGBoost Documentation," [Online]. Available: https://xgboost.readthedocs.io/

[7] scikit-learn developers, "Cross-validation: evaluating estimator performance," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/cross_validation.html

[8] [Dataset source], "College_placement.csv," [Repository]. [Online]. Available: [Pending — insert the exact dataset repository/source URL used by the team].

APPENDIX

A.1 Full source code: [Pending — insert the team's GitHub/Colab repository link.]

A.2 Complete weekly PBL log and mentor sign-offs: see Table 3.1; extend here if the team's actual log runs longer than the five-row summary in the main text.

A.3.0 Full baseline training script (reference listing)

```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)


# --- 1. Load and inspect ---
df = pd.read_csv("College_placement.csv")
print("Shape:", df.shape)
print("Missing values per column:")
print(df.isnull().sum())
print(df.describe())


# --- 2. Clean and split features/target ---
df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")
X = df.drop(columns=["Placement"])
y = df["Placement"]


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)


# --- 3. Duplicate / leakage check (Iteration 2) ---
n_dupes = df.duplicated().sum()
print(f"Exact duplicate rows in dataset: {n_dupes}")
overlap = pd.merge(X_train.reset_index(), X_test.reset_index(), how="inner",
                   on=list(X.columns))
print(f"Feature-identical rows shared across train/test split: {len(overlap)}")
```

```
# --- 4. Baseline model: Random Forest ---
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

print("Random Forest baseline (single split):")
print(" Accuracy :", accuracy_score(y_test, y_pred))
print(" Precision:", precision_score(y_test, y_pred))
print(" Recall   :", recall_score(y_test, y_pred))
print(" F1-score :", f1_score(y_test, y_pred))
print(" Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
```

--- 5. Cross-validation (5-Fold Stratified CV) ---

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(rf, X, y, cv=cv, scoring="accuracy")
print("5-fold CV accuracy: mean=%.4f std=%.4f" % (cv_scores.mean(), cv_scores.std()))
```

--- 6. Comparison models (completed evaluation) ---

```
models = {
    "LogisticRegression": LogisticRegression(max_iter=1000),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "DecisionTree": DecisionTreeClassifier(random_state=42),
    "RandomForest": rf,
```

"XGBoost": XGBClassifier(n_estimators=200, max_depth=6, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, random_state=42),

```
}
for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=cv, scoring="accuracy")
    print(f"{name}: mean CV accuracy = {scores.mean():.4f} (std {scores.std():.4f})")

# --- 7. Feature importance (Random Forest) ---
importances = pd.Series(rf.feature_importances_, index=X.columns)
print(importances.sort_values(ascending=False))
```

A.3 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
R SABARISH KUMAR	50%	50%	Contributed to ML development, system integration, implementation, testing, and documentation.
N DINESH SAI KARTHIK	50%	50%	Contributed to application development, data handling, implementation, testing, and documentation.